

Isabelle/HOL Learning Notes

Contents

Background	1
Setup	2
Term	2
Lemma Example	2
Rationals	3
Summation, Factorial, Exponentiation	4
Defining the Explicit Formula	4
Checkpoint & Proof Strategy	4
Proving the Base Case	5
Aside: Finding Existing Useful Theorems	6
Proving the Split Successor Recurrence Lemma	6
The Stirling Identity	6
Proving the Stirling Shift Lemma	7
Using the Stirling Identity	7
Reindexing the Stirling Double Sum Formula	8
Combining Summands Proof	9
Final Proof of Successor Recurrence	9
The Induction Theorem	10
Final Theorem	11

Background

These notes document an AI-tutor-assisted Isabelle/HOL learning project developed with Codex. The project formalises the explicit Stirling-number formula for the Akiyama–Tanigawa triangle that appears in my Math Stack-Exchange answer to this question. The triangle is also recorded as OEIS A371763.

The theory proves the explicit formula directly from the defining recurrence. It does not formalise my original complex-analytic derivation or attempt the (very difficult) problem of classifying all non-trivial zeros.

My primary goal was to learn the fundamentals of machine-checked proof development in Isabelle. By completing the proof, I developed a working understanding of the mathematical purpose of each stage, the structure of the induction argument, and the role played by the principal Isabelle commands and library lemmas.

Some lower-level proof-engineering details remain areas for further consolidation:

- In “Reindexing the Stirling Double Sum Formula” and “Final Proof of Successor Recurrence”, I want to revisit the individual `have` and `proof` blocks.
- In “The Induction Theorem”, I want to become more fluent with Isabelle’s induction syntax.
- In “Final Theorem”, I want to consolidate my understanding of the interaction between `assumes`, `shows`, `using`, and proof methods such as `by`.

Setup

- Create a new file in Isabelle and save as, e.g. `Stirling_Formula.thy`.
- Next add the header:

```
theory Stirling_Formula
  imports Main
begin
```

Note – ``theory Stirling_Formula`` must match the filename.

- Isabelle checks continuously as you type.
- Press `⌘S` to save the file.

Term

- We’ll need to use the Stirling number import, so change the file to

```
theory Stirling_Formula
  imports "HOL-Combinatorics.Stirling"
begin
  term Stirling      (* outputs: "Stirling" :: nat => nat => nat *)
end
```

which also imports `Main` as a transitive dependency.

- Here `term` is an inspection command, and clicking “`Stirling`” will show the Output-panel result recorded in the above comment; this loosely represents $f : \mathbb{N}^2 \rightarrow \mathbb{N}$. Note, however, that `=` is right-associative meaning `Stirling :: nat => (nat => nat)` so an input $n \in \mathbb{N}$ returns a function which takes another $k \in \mathbb{N}$ which then outputs the result $S_n^{(k)} \in \mathbb{N}$. This is an example of *currying*, in which a function regarded mathematically as taking two arguments is represented as a function that takes n and returns another function taking k . It reminds me of nested lambda functions such as `n -> k -> ...`.
- You can compute a Stirling number by adding e.g. value `"Stirling 0 0"` and clicking the command returns `"1" :: "nat"` where the quotation marks represent parsed terms, so neither quoted item is a string because `1` is a term while `nat` is a type.

Lemma Example

- We can add a lemma with

```
lemma learning_Stirling_00: "Stirling 0 0 = 1"
```

Here `Stirling 0 0 = 1` is the proposition to be proved which means $(\text{Stirling } 0) \ 0 = 1$, or equivalently $f(0)(0) = 1$ or $S_0^{(0)} = 1$.

- At this point the theory lacks proof of the lemma, so the `end` keyword has a red underline with a “Bad context” error in the output.
- Clicking the “Proof state” checkbox, and on the lemma, gives the Output-result:

```

proof (prove)
goal (1 subgoal):
  1. Stirling 0 0 = 1
Auto solve_direct: the current goal can be solved directly with
  HOL.nitpick_simp(455): Stirling 0 0 = 1
  Stirling.Stirling.simps(1): Stirling 0 0 = 1
  Stirling.Stirling_same: Stirling ?n ?n = 1

```

where the `Auto solve_direct:` is advisory. The listed proof approaches are `HOL.nitpick_simp(455)`, `Stirling.Stirling.simps(1)`, and `Stirling.Stirling_same`. These are existing facts and theorems from various dependencies. So I chose `Stirling.Stirling.simps(1)` because this matches exactly what we're trying to prove in a descriptive way that `HOL.nitpick_simp(455)` lacks.

- So we can add this proof to our theory:

```

begin
  lemma learning_Stirling_00: "Stirling 0 0 = 1"
  by (rule Stirling.Stirling.simps(1))
end

```

where `by` starts a short proof intended to finish the goal immediately and `rule` applies the existing proved fact.

Rationals

The recurrence is

$$a_{0,j} = \frac{1}{j}, \quad a_{i,j} = j(a_{i-1,j} - a_{i-1,j+1}) \quad (i \geq 1, j \geq 1).$$

For primitive recursion in Isabelle, rewrite it in successor form:

$$a_{0,j} = \frac{1}{j}, \quad a_{i+1,j} = j(a_{i,j} - a_{i,j+1}).$$

The two forms are equivalent after replacing i in the original recursive equation by $i + 1$. Isabelle represents the successor index $i + 1$ structurally as `Suc i` which is equivalent to $i + 1$.

- Typing term `"(1 :: rat) / 2"` inspects a `rat` example, and exposes the missing import `HOL.Rat` which I needed to add to the theory `imports` list.
- To define $a_{i,j}$ we'll need to define a function `a :: nat ⇒ nat ⇒ rat` with $i, j \in \mathbb{N}$ and $a_{i,j} \in \mathbb{Q}$.
- $j \in \mathbb{N}$ but in the recurrence arithmetic we will need to work with reciprocals of j so we'll need a way to convert j to a rational. We can use Isabelle's `of_nat` for this, e.g.

```

term "of_nat :: nat ⇒ rat"      (* outputs: "rat_of_nat" :: "nat ⇒ rat" *)

```

where Isabelle resolved the overloaded conversion `of_nat` to the concrete rational conversion `rat_of_nat`.

- So $a_{i,j} = 1/j$ can be written in Isabelle as `a 0 j = 1 / of_nat j`, where Isabelle will specialise `1` and `/` to the rationals since `a` is defined to return `rat`, and Isabelle resolves `of_nat` to `rat_of_nat`.
- So, we can define the full recurrence relation using `primrec` or “primitive recursion”. For natural numbers, `primrec` defines a function using a base case for `0`, and a successor case for `Suc i`, expressed using the already-defined value at the smaller index i . `primrec` is more restricted than Isabelle's general `fun` command.

```

primrec a :: "nat ⇒ nat ⇒ rat" where
  "a 0 j = 1 / of_nat j"
| "a (Suc i) j = of_nat j * (a i j - a i (Suc j))"

```

If you click the code you should see `a :: nat ⇒ nat ⇒ rat` in the output.

- Next add `thm a.simps` (simplifications) to show the generated equations:

```

a 0 ?j = 1 / rat_of_nat ?j
a (Suc ?i) ?j = rat_of_nat ?j * (a ?i ?j - a ?i (Suc ?j))

```

so Isabelle generated two proved rewrite rules `a.simps(1)` and `a.simps(2)`.

- The equations are now machine-checked facts generated from the accepted definition.
- Typing e.g. `value "a 0 2"` results in `"1 / 2" :: "rat"`.

Summation, Factorial, Exponentiation

- Typing `\<Sum>` produces $\sum$, so we can type e.g. `value "\sum l=0..3. l :: nat"` to output `"6" :: "nat"`.
- Typing term `fact` and clicking it gives the output `"fact" :: "nat => 'a"` where `'a` is a *type variable* and pronounced “type alpha” which is a type that can be specialised to the surrounding context. Ordinarily $n!$ with $n \in \mathbb{N}$ returns the natural number $n! = n(n-1)(n-2)\dots 2 \cdot 1$, but `'a` means Isabelle can interpret `fact` as returning `nat` or `int` or `rat` or `real`, for example. The `'` of `'a` marks the presence of a type variable.
- In Isabelle $^$ denotes exponentiation where the exponent is a natural number. Typing `value "(-1 :: rat) ^ 3"` gives output `"- 1" :: "rat"`.
- So using these we can convert the summand

$$(-1)^\ell \frac{\ell! S_i^{(\ell)}}{j + \ell}$$

with `"(-1 :: rat) ^ l * (fact l :: rat) * of_nat (Stirling i l) / of_nat (j + l)"`. Here $S_i^{(\ell)}$ denotes the Stirling number of the second kind; Isabelle represents it as `Stirling i l`.

- Adding term `"(-1 :: rat) ^ l * (fact l :: rat) * of_nat (Stirling i l) / of_nat (j + l)"` and clicking it confirms Isabelle is using the summand type `"(- 1) ^ l * fact l * rat_of_nat (Stirling i l) / rat_of_nat (j + l)" :: "rat"`, which is useful because it also shows how it was able to infer types. Either the annotation on `(-1 :: rat)` or `(fact l :: rat)` can be removed, provided the other remains to supply enough rational context.
- Combining the finite sum with the summand, we obtain: term `"\sum l=0..i. (-1 :: rat) ^ l * fact l * of_nat (Stirling i l) / of_nat (j + l)"`.
- Finally, to obtain the full formula

$$(-1)^i \sum_{\ell=0}^i (-1)^\ell \frac{\ell! S_i^{(\ell)}}{j + \ell},$$

add the indicated coefficient, with parentheses around the whole sum: term `"(-1 :: rat)^i * (\sum l=0..i. (-1)^l * fact l * of_nat (Stirling i l) / of_nat (j + l))"`.

Defining the Explicit Formula

- The last bullet point in the previous section is the full definition of our explicit formula, which we can define in Isabelle using `definition` since it's not a recurrence relation:

```
definition closed_form :: "nat => nat => rat" where
  "closed_form i j = (-1 :: rat)^i * (\<Sum>l=0..i. (-1)^l * fact l * of_nat (Stirling i l) / of_nat (j + l))"
```

- Typing `thm closed_form_def` should output the definition.

Checkpoint & Proof Strategy

- So the function definitions are currently:

```
theory Stirling_Formula
  imports
    "HOL-Combinatorics.Stirling"
    "HOL.Rat"
begin

primrec a :: "nat => nat => rat" where
  "a 0 j = 1 / of_nat j"
| "a (Suc i) j = of_nat j * (a i j - a i (Suc j))"
```

```

definition closed_form :: "nat ⇒ nat ⇒ rat" where
  "closed_form i j =
    (-1 :: rat)^i *
    (\<Sum>l=0..i.
      (-1)^l * fact l * of_nat (Stirling i l) / of_nat (j + l))"

end

```

- `closed_form` is an explicit formula, but our proof strategy will be to show that it satisfies the same base equation and successor recurrence as `a`, therefore proving the two agree (by an induction proof) on the intended domain $j \geq 1$.

On the intended domain $j \geq 1$, we want to establish

$$\text{closed_form}(0, j) = \frac{1}{j}$$

and

$$\text{closed_form}(i + 1, j) = j (\text{closed_form}(i, j) - \text{closed_form}(i, j + 1)).$$

In Isabelle syntax, the desired equations have the forms

```

closed_form 0 j = 1 / of_nat j

closed_form (Suc i) j =
  of_nat j * (closed_form i j - closed_form i (Suc j))

```

- These are not additional definitions, but proof obligations that must be derived from `closed_form_def`. Once they are proved, induction on i can be used to show that `a` and `closed_form` agree.
- We need to make sure the domains for i, j are correctly defined and dealt with in Isabelle. The types already dictate $i, j :: \text{nat}$ so $i, j \geq 0$, and the summation index $l :: \text{nat}$ is bounded by $0..i$. The intended theorem, however, is restricted to $j \geq 1$.
- Isabelle/HOL functions are total, so rational division by zero is defined and both functions have values at $j = 0$. At this checkpoint, however, we have not proved that `a` and `closed_form` agree at any indices.
- The original mathematical problem has the intended domain $j \geq 1$, so the final theorem will assume $0 < j$. As we develop the intermediate base-case and recurrence lemmas, we will determine precisely where that positivity assumption is required.
- My original proof used Complex Analysis, in particular Cauchy's integral formula and residues to derive the explicit formula, whereas the Isabelle proof uses induction. The key Stirling identity is

$$S_{i+1}^{(n)} = nS_i^{(n)} + S_i^{(n-1)} \quad (n \geq 1),$$

with the boundary case $S_{i+1}^{(0)}$ handled separately.

Proving the Base Case

- As discussed in the previous section we need to prove that `closed_form 0 j` satisfies the same base equation and successor as `a`. For the base equation:

```

lemma closed_form_0:
  "closed_form 0 j = 1 / of_nat j"
  unfolding closed_form_def (* 1. replace closed_form with defn *)
  apply simp                (* 2. use simplify to prove it *)
  done

```

If you check the output at comment 1 you will see the general formula with $i = 0$. At comment 2 we apply Isabelle's simplification (`simp`) rules to try to show equality, i.e. prove that `closed_form 0 j = 1 / of_nat j`. I noticed that `apply` and `done` show up red in the editor, but this is common in Isabelle. An actual error is designated by a wavy red underscore.

- So, at $i = 0$, the sum contains only $\ell = 0$. Since $(-1)^0 = 1$, $0! = 1$, and $S_0^{(0)} = 1$, it simplifies to $1/j$. No assumption $j > 0$ is needed for this lemma: at $j = 0$, both sides equal 0 under Isabelle’s totalised rational division.*

Aside: Finding Existing Useful Theorems

Using `find_theorems` searches for existing relevant theorems, e.g. `find_theorems name:atLeast_Suc_atMost` listed:

```
find_theorems
  name: "atLeast_Suc_atMost"

found 6 theorem(s):
...
Set_Interval.comm_monoid_add_class.sum.atLeast_Suc_atMost:
  ?m ≤ ?n ==> sum ?g {?m..?n} = ?g ?m + sum ?g {Suc ?m..?n}
...
```

- This theorem requires $m \leq n$ and deals with a sum

$$\sum_{i=m}^n g(i) = g(m) + \sum_{i=m+1}^n g(i).$$

which we’ll need in the following section.

Proving the Split Successor Recurrence Lemma

- We now focus on the Successor Recurrence, and put it into a “split” form by separating the $\ell = 0$ term of the sum. We separate the $\ell = 0$ term because the Stirling recurrence involving $S_i^{(\ell-1)}$ is used only for $\ell \geq 1$. The boundary term is handled separately using $S_{i+1}^{(0)} = 0$.
- For the successor recurrence we use `Suc i`:

```
lemma closed_form_Suc_split:
  "closed_form (Suc i) j =
    (-1 :: rat)^(Suc i) *
    (of_nat (Stirling (Suc i) 0) / of_nat j + (\<Sum>l=1..(Suc i).
      (-1)^l * fact l * of_nat (Stirling (Suc i) l) / of_nat (j + l)))"

unfolding closed_form_def

apply (simp only: sum.atLeast_Suc_atMost)  (* 1 *)
apply simp                                (* 2 *)
done
```

- Using `find_theorems name:atLeast_Suc_atMost` reveals the useful existing theorem `sum.atLeast_Suc_atMost` from the example in the previous section, which we apply above. Note that `simp only:` restricts simplification to the supplied rule, together with Isabelle’s basic simplifier. The output shows what `closed_form_Suc_split` achieves before adding other simplifications.
- After the first `apply`, it turned out Isabelle found two equivalent forms, which was resolved after the second `apply simp` which found them to be equivalent.
- This proves that the new split version of is equivalent to the original version `closed_form_def`.

Light Bulb Moment: A useful realisation is that Isabelle machine-checks every transformation before we use its result in the next step. In a handwritten proof, we might simply write “split off the $\ell = 0$ term”, treating this as an immediate finite-sum identity. In Isabelle, we explicitly justify that step using the proved theorem `sum.atLeast_Suc_atMost`. The resulting equality can then be recorded as the named lemma `closed_form_Suc_split` and used later.

The Stirling Identity

- Next we need to find out what Isabelle knows about the Stirling numbers. Typing `thm Stirling.simps` outputs:

```

Stirling 0 0 = 1
Stirling 0 (Suc ?k) = 0
Stirling (Suc ?n) 0 = 0
Stirling (Suc ?n) (Suc ?k) = Suc ?k * Stirling ?n (Suc ?k) + Stirling ?n ?k

```

where the last one is exactly the required identity

$$S_{n+1}^{(k+1)} = (k+1)S_n^{(k+1)} + S_n^{(k)} \quad (n \geq 0). \quad (1)$$

- Since the indices differ slightly from our original indices, we will need to set $\ell = k + 1$ in the summand

$$(-1)^\ell \frac{\ell! S_{i+1}^{(\ell)}}{j + \ell} \mapsto (-1)^{k+1} \frac{(k+1)! S_{i+1}^{(k+1)}}{j + k + 1}.$$

Proving the Stirling Shift Lemma

- To use Isabelle's Stirling theorems we need to rewrite the `closed_form Suc i j` as a sum with the above summand with $\ell = k + 1$. This is easily done; I reused the Isabelle code from the Proving the Split Successor Recurrence Lemma section:

```

"closed_form (Suc i) j =
  (-1 :: rat)^(Suc i) *
  (of_nat (Stirling (Suc i) 0) / of_nat j + (\<Sum>k=0..i.
    (-1)^(Suc k) * fact (Suc k) * of_nat (Stirling (Suc i) (Suc k)) / of_nat (j + Suc k)))"

```

- The isolated $S_{i+1}^{(0)}$ term is identically zero, so we obtain

```

lemma closed_form_Suc_shift:
  "closed_form (Suc i) j =
    (-1 :: rat)^(Suc i) *
    (\<Sum>k=0..i. (-1)^(Suc k) * fact (Suc k)
      * of_nat (Stirling (Suc i) (Suc k)) / of_nat (j + Suc k))"
  unfolding closed_form_Suc_split

  (* use the 3rd theorem output of thm Stirling.simps *)
  apply (simp only: Stirling.simps(3))

  (* apply basic simplifications *)
  apply (simp only: of_nat_0 div_0 add_0)

  (* apply reindexing step *)
  apply (simp only: One_nat_def)    (* rewrites 1 = Suc 0 for next step *)
  apply (simp only: sum.atLeast_Suc_atMost_Suc_shift)
  apply (simp only: comp_def)      (* to collapse o composition synta *)

```

- The `sum.atLeast_Suc_atMost_Suc_shift` apply was noted in the output of `find_theorems` name: "`atLeast_Suc_atMost`" and is simply a reindexing theorem on finite sums. The surrounding `applies` are to a kind of boilerplate in a way to nudge the proof given various resulting syntaxes.

Light Bulb Moment: In the above lemma we actively inspect the remaining goal mismatch (from the output), choose a mathematically justified transformation, and give Isabelle a theorem that certifies it. Using `apply` in this way eventually resolves all mismatches until the subgoal is met/proved.

Using the Stirling Identity

- Next we can replace $S_{i+1}^{(k+1)}$ with the identity in (1):

```

lemma closed_form_Suc_stirling:
  "closed_form (Suc i) j =
    (-1 :: rat)^(Suc i) *
    (\<Sum>k=0..i. (-1)^(Suc k) * fact (Suc k)
      * of_nat (Suc k * Stirling i (Suc k) + Stirling i k) / of_nat (j + Suc k))"
  unfolding closed_form_Suc_shift

```

```
(* apply the 4th theorem of thm Stirling.simps *)
apply (simp only: Stirling.simps(4))
done
```

Light Bulb Moment: unfolding `closed_form_Suc_shift` rewrites the LHS using `closed_form_Suc_shift`, with the output proof goal mismatch being between the original Stirling number and the new Stirling identity, which we then go ahead and prove using various `appls`. In the goal output you'll notice `LHS = RHS`.

- Next a proof of splitting into two sums:

```
lemma closed_form_Suc_two_sums:
  "closed_form (Suc i) j =
  (-1 :: rat)^(Suc i) *
  ((\<Sum>k=0..i. (-1)^(Suc k) * fact (Suc k)
    * of_nat (Suc k * Stirling i (Suc k)) / of_nat (j + Suc k))
  + (\<Sum>k=0..i. (-1)^(Suc k) * fact (Suc k)
    * of_nat (Stirling i k) / of_nat (j + Suc k)))"
  unfolding closed_form_Suc_stirling

  apply (simp only: of_nat_add distrib_left add_divide_distrib sum.distrib)
done
```

- Mathematically we want to reindex the first sum over k to range over $[0, \dots, i]$ instead of $[1, \dots, i + 1]$ to match the second sum; whereas currently we have $\sum_{k=0}^i$ and use `Suc k` in the summand, so $k + 1$.
- To achieve this we will make use of `sum_shift_lb_Suc0_0` and `sum.atLeast0_atMost_Suc` (type `thm sum_shift_lb_Suc0_0 sum.atLeast0_atMost_Suc` to see)

Reindexing the Stirling Double Sum Formula

- See note in Background section for follow up work on this section.

```
(* reindex the first sum to match the second indexing *)
lemma closed_form_Suc_aligned:
  "closed_form (Suc i) j =
  (-1 :: rat)^(Suc i) *
  ((\<Sum>k=0..i. (-1)^k * fact k
    * of_nat (k * Stirling i k) / of_nat (j + k))
  + (\<Sum>k=0..i. (-1)^(Suc k) * fact (Suc k)
    * of_nat (Stirling i k) / of_nat (j + Suc k)))"
  proof -
    let ?F = "\k.
      (-1 :: rat)^k * fact k *
      of_nat (k * Stirling i k) / of_nat (j + k)"
    have F0: "?F 0 = 0"
      by simp
    have Ftop: "?F (Suc i) = 0"
      by simp
    have Fshift:
      "\<Sum>k=0..i. ?F (Suc k) = (\<Sum>k=0..i. ?F k)"
    proof -
      have shift:
        "\<Sum>k=0..i. ?F (Suc k) = (\<Sum>k=Suc 0..Suc i. ?F k)"
        using sum.atLeast_Suc_atMost_Suc_shift[of ?F 0 i, symmetric]
        by (simp only: comp_def)
      have lower:
        "\<Sum>k=Suc 0..Suc i. ?F k = (\<Sum>k=0..Suc i. ?F k)"
        apply (rule sum_shift_lb_Suc0_0[where f="?F" and k="Suc i"])
        apply (rule F0)
        done
      have upper:
        "\<Sum>k=0..Suc i. ?F k = (\<Sum>k=0..i. ?F k)"
        by (simp only: sum.atLeast0_atMost_Suc Ftop add_0_right)
      show ?thesis
        by (simp only: shift lower upper)
    qed
  show ?thesis
```

```

unfolding closed_form_Suc_two_sums
by (simp only: Fshift)
qed

```

Combining Summands Proof

- The next step is to combine the two finite sums into one:

```

lemma closed_form_Suc_combined:
  "closed_form (Suc i) j =
    (-1 :: rat)^(Suc i) *
      (\<Sum>k=0..i. (-1)^k * fact k
        * of_nat (Stirling i k) * (of_nat k / of_nat(j + k) - of_nat (Suc k) / of_nat (j + (Suc k))))"
unfolding closed_form_Suc_aligned
apply (simp only: of_nat_mult fact_Suc power_Suc)      (* use multiplicative property of rat_of_nat *)
apply (simp only: right_diff_distrib sum_subtractf)    (* distribute: a(b-c) = ab - ac *)
apply (simp only:      (* fix ordering issues *)
  times_divide_eq_right
  ac_simps
  mult_minus_left
  mult_1
  sum_negf
  diff_conv_add_uminus)
apply (simp only:
  mult_minus_right
  divide_minus_left
  sum_negf)
apply (simp only:
  distrib_left
  mult_minus_right
  minus_add_distrib
  minus_minus)
done

```

Final Proof of Successor Recurrence

- This proves the successor notation.
- See note in Background section for follow up work on this section.

```

lemma closed_form_Suc:
  assumes j_pos: "0 < j"
  shows "closed_form (Suc i) j =
    of_nat j * (closed_form i j - closed_form i (Suc j))"
proof -
  have denom: "of_nat (j + k) ≠ (0 :: rat)" for k
    using j_pos by simp
  have denom_Suc: "of_nat (j + Suc k) ≠ (0 :: rat)" for k
    by simp
  have fraction_identity:
    "(of_nat k :: rat) / of_nat (j + k) -
      of_nat (Suc k) / of_nat (j + Suc k) =
      - (of_nat j) *
        (1 / of_nat (j + k) - 1 / of_nat (j + Suc k))"
  for k
proof -
  have first:
    "(of_nat k :: rat) / of_nat (j + k) =
      1 - of_nat j / of_nat (j + k)"
  proof -
    have "(of_nat k :: rat) / of_nat (j + k) =
      (of_nat (j + k) - of_nat j) / of_nat (j + k)"
      by (simp only: of_nat_add add_diff_cancel_left')
    also have "... =
      of_nat (j + k) / of_nat (j + k) -
      of_nat j / of_nat (j + k)"
      by (rule diff_divide_distrib)

```

```

    also have "... =
      1 - of_nat j / of_nat (j + k)"
    by (simp add: denom[of k])
    finally show ?thesis .
qed
have second:
  "(of_nat (Suc k) :: rat) / of_nat (j + (Suc k)) =
    1 - of_nat j / of_nat (j + (Suc k))"
proof -
  have "(of_nat (Suc k) :: rat) / of_nat (j + (Suc k)) =
    (of_nat (j + (Suc k)) - of_nat j) / of_nat (j + (Suc k))"
  by (simp only: of_nat_add add_diff_cancel_left')
  also have "... =
    of_nat (j + (Suc k)) / of_nat (j + (Suc k)) -
    of_nat j / of_nat (j + (Suc k))"
  by (rule diff_divide_distrib)
  also have "... =
    1 - of_nat j / of_nat (j + (Suc k))"
  by (simp add: denom_Suc[of k])
  finally show ?thesis .
qed
show ?thesis
  apply (simp only: first second)
  apply (simp add: algebra_simps)
  done
qed
show ?thesis
  apply (subst closed_form_Suc_combined)
  apply (simp only: fraction_identity)
  apply (simp only: closed_form_def)
  apply (simp only: power_Suc) (* rearrange signs, products and sums *)
  apply (simp only: add_Suc_right add_Suc_shift)
  apply (simp add: algebra_simps sum_subtractf)
  apply (simp add:
    sum_distrib_left
    sum_distrib_right
    algebra_simps)
  done
qed

```

The Induction Theorem

- See note in Background section for follow up work on this section.

```

theorem a_eq_closed_form:
  assumes j_pos: "0 < j"
  shows "a i j = closed_form i j"
  using j_pos
proof (induction i arbitrary: j)
case 0
then show ?case
  by (simp add: closed_form_0)
next
case (Suc i)
have IH_j: "a i j = closed_form i j"
  by (rule Suc.IH[OF Suc.prem1])
have IH_Suc_j: "a i (Suc j) = closed_form i (Suc j)"
  using Suc.IH[of "Suc j"] by simp
show ?case
  by (simp add:
    IH_j
    IH_Suc_j
    closed_form_Suc[OF Suc.prem1])
qed

```

Final Theorem

And we can state the final result as a theorem:

```
theorem a_explicit_formula:
  assumes j_pos: "0 < j"
  shows "a i j =
    (-1 :: rat)^i *
    (\<Sum>l=0..i.
      (-1)^l * fact l * of_nat (Stirling i l) / of_nat (j + l))"
  using a_eq_closed_form[OF j_pos]
  by (simp only: closed_form_def)

thm a_explicit_formula
```

which outputs:

```
0 < ?j ==>
a ?i ?j = (- 1) ^ ?i * (\<Sum>l = 0..?i. (- 1) ^ l * fact l * rat_of_nat (Stirling ?i l) / rat_of_nat (?j + l))
```